02. 설치 및 환경 설정

이 장에서는 Hermes Agent를 설치하고 LLM과 연동하는 과정을 단계별로 안내합니다. 로컬 LLM(Ollama)과 클라우드 API(Claude, GPT 등) 모두 사용할 수 있습니다.

이 장에서 다루는 내용

- 시스템 요구사항
- 설치하기
- LLM 연동
- 설정 파일 이해하기

02-1. 시스템 요구사항

- 지원 플랫폼
- 필수 소프트웨어
- 하드웨어 권장사항
- 책 집필에 사용한 환경

지원 플랫폼

- **Linux**: 대부분의 배포판 지원
- **macOS**: Intel과 Apple Silicon 지원
- **Windows**: WSL2가 공식 검증 경로(road-tested). 네이티브 Windows는 초기 베타 단계(early beta)
- **Android**: Termux로 일부 기능 사용 가능

원라인 설치 스크립트는 Linux·macOS·WSL2에서 동일하지만, OS와 버전에 따라 실제 설치 경험과 사용 편의성은 차이가 있습니다.

환경	비고
Linux	원라인 설치가 그대로 작동
macOS (Apple Silicon, Intel Monterey 이상)	Homebrew와 Xcode Command Line Tools 필요. Homebrew bottle 정상 지원
macOS Intel (Big Sur 이하)	Homebrew Tier 3 강등으로 bottle 미제공 — MacPorts로 우회(「설치하기」 절 참고). 새 TUI 변형은 macOS 12 SDK가 필요해 빌드 불가
WSL2 (Windows)	공식 검증 경로. 단, WSL2 세션 종료 시 cron·gateway가 함께 정지. 파일은 `/mnt/c/` 보다 Linux 홈 디렉터리에 두는 편이 빠름
Native Windows	공식 표기 early beta(초기 베타). subprocess·경로·non-ASCII 콘솔 출력이 아직 거침. 대시보드 채팅 창(pane)은 PTY 기반이라 WSL2 필요
Termux (Android)	`.[termux]` 전용 extras 사용. `.[all]` 은 Android와 호환되지 않는 음성 의존성을 포함해 사용 불가

필수 소프트웨어

Hermes Agent의 원라인 설치 스크립트가 대부분의 의존성을 자동으로 설치하지만, **Git**은 사전에 설치되어 있어야 합니다.

설치 스크립트가 자동으로 처리하는 항목:

- **Python 3.11 이상**: uv 패키지 매니저로 관리(필요 시 `~/.hermes/bin`에 전용 설치)
- **Node.js 22 LTS**: 브라우저 자동화 및 WhatsApp 연동에 필요. 시스템 Node가 요구 버전(`^20.19 || >=22.12`)에 못 미치면 `~/.hermes/node/`에 관리형 Node를 따로 설치
- **ripgrep**: 빠른 파일 검색 (선택, 없으면 grep으로 대체)
- **ffmpeg**: 음성 모드에 필요 (선택)

하드웨어 권장사항

Hermes Agent는 Python CLI에 Node.js 기반 구성 요소(TUI·웹 UI)가 결합된 구조입니다. 공식 문서는 하드웨어 최소 사양을 별도로 명시하지 않으며, 클라우드 API 모드 기준으로 다음 정도면 사용에 무리가 없습니다.

- **CPU**: 듀얼 코어 이상
- **RAM**: 4GB 이상 (클라우드 API 사용 가정)
- **디스크**: 2GB 이상 (설치 및 데이터용)

로컬 LLM을 같은 머신에서 돌린다면

로컬에서 LLM을 직접 실행하려면 추가 리소스가 필요합니다. 예를 들어 Llama 3.1 8B(Q4_K_M)는 약 5GB의 RAM/VRAM을, Qwen3 14B(Q4_K_M)는 약 9.3GB를 사용합니다. 자세한 환경 구성과 모델 비교는 부록「로컬 LLM 가이드」를 참고하세요.

책 집필에 사용한 환경

다음은 이 책을 쓰면서 사용한 환경입니다. 일부 실습은 이 환경에서 우회 설치로 진행했고, 새 TUI(`hermes --tui`)처럼 macOS 12 SDK 이상이 필요한 기능은 Monterey 이상 또는 다른 머신에서 확인했습니다.

Hermes Agent 구동 머신:

- macOS Big Sur 11.7 (Intel Core M, 8GB RAM)
- 디스크 여유 공간: 213GB
- 빌드·실행 우회는「설치하기」의 macOS Big Sur Tier 3 우회 TIP 참고

기본 LLM 구성:

- OpenRouter를 통한 클라우드 모델 —「첫 대화 실습」절은 무료 모델(`nvidia/nemotron-3-super-120b-a12b:free`), 이후 실습은 필요에 따라 유료 모델로 전환

- LAN에 있는 별도 머신의 Ollama 서버도 사용 가능 — 부록 「로컬 LLM 가이드」와 로컬 LLM을 다루는 일부 장에서 사용

구형 Intel Mac에서는 패키지·모듈 설치 과정에 우회가 필요한 부분이 있으니 「설치하기」 절의 macOS Big Sur Tier 3 우회 TIP을 함께 보세요.

02-2. 설치하기

> ⚠️ ***부분 검증**: 원라인 설치는 macOS Intel Mac에서 직접 테스트했습니다(v0.14.0). 데스크톱 앱 설치는 v0.16.0 공식 문서·소스(`scripts/install.sh` , 데스크톱 빌드)를 대조해 정리했고 실행 테스트는 하지 않았습니다. PyPI, Zed ACP, Native Windows 베타 내용은 소스 코드(`scripts/install.sh` , PowerShell 인스톨러)와 릴리스 노트를 대조해 정리했습니다. 일반 Linux/Ubuntu 설치 절은 v0.17.0 소스(`scripts/install.sh`)와 실제 Ubuntu 데스크톱 설치 관찰을 대조해 정리했습니다.*

- 설치 방법 선택
- 데스크톱 앱
- 원라인 설치
- PyPI 패키지 설치
- 수동 설치
- 플랫폼별 설치와 주의사항
  - macOS
  - Native Windows (베타)
  - Termux / Android
  - 일반 Linux / Ubuntu 데스크톱 설치 (비root)
  - Linux root 설치와 FHS 레이아웃
- Zed 에디터 연동
- 설치 확인과 초기 설정
- 선택 기능 설치 (lazy-install)
- 기존 환경 이전 — hermes backup / hermes import
- 버전별 변경 이력

설치 방법 선택

환경에 맞는 설치 방법을 고릅니다.

환경	권장 방법
GUI로 채팅·미리보기·파일 탐색을 쓰고 싶을 때	데스크톱 앱
새 환경에서 Python·Node·ripgrep·ffmpeg를 한 번에 준비	원라인 설치
Python 환경이 이미 있고 Hermes만 추가로 설치할 때	PyPI 설치(`pip` 또는 `uv tool install`)
Zed 에디터에서 연동해 쓰는 경우	본체 설치 후 Zed ACP 등록
Windows에서 WSL 없이 네이티브로 실행	Native Windows 베타
Android/Termux	Termux 방법
자동 스크립트 없이 단계별로 직접 설치	수동 설치

각 설치 방법의 자세한 절차는 아래 절에서 다룹니다. 설치 후 점검과 초기 설정은 모든 방법에 공통으로 적용되므로 「설치 확인과 초기 설정」 절에 모아 두었습니다.

데스크톱 앱

터미널이 익숙하지 않다면, 가장 쉬운 길은 웹사이트에서 **데스크톱 설치 관리자**를 내려받아 실행하는 것입니다. 이 관리자가 CLI와 데스크톱 앱을 함께 설치합니다.

```
https://hermes-agent.nousresearch.com/desktop
```

데스크톱 앱(macOS·Windows·Linux)은 CLI와 **같은** 에이전트 코어를 GUI로 감싼 것입니다. 같은 설정·API 키·세션·스킬·메모리를 공유하므로, 데스크톱에서 한 작업이 CLI에도 그대로 보이고 그 반대도 됩니다. 스트리밍 도구 출력, 좌우 분할 미리보기, 파일 브라우저, 음성, 크론, 프로필을 한 화면에서 다룹니다. 설치 후 CLI에서 `hermes desktop` 으로 앱을 빌드·실행할 수도 있습니다.

> ⚠️ ***신생 앱 안내**: 데스크톱 앱은 2026년 5월 말 공개된 신규 구성요소로, 온보딩·설정 화면이 릴리스마다 빠르게 바뀌고 있습니다. 이 책은 안정적으로 작동하고 소스로 검증하기 쉬운 **CLI를 기준**으로 설명합니다. 데스크톱 앱의 화면별 절차는 안정화된 뒤 별도로 다룰 예정입니다.*

원라인 설치

가장 간단한 설치 방법은 공식 설치 스크립트입니다.

```
curl -fsSL https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
```

스크립트가 다음을 자동으로 수행합니다.

1. Python 3.11 환경 구성(uv 패키지 매니저 사용)
2. Node.js 22 LTS 준비(시스템 Node가 요구 버전에 못 미치면 관리형 Node를 따로 설치)
3. ripgrep·ffmpeg 설치 여부 확인(선택, 거절 가능)
4. 저장소 클론 및 의존성 설치(해시 검증 lockfile 우선)
5. Node.js 의존성 및 Playwright 브라우저 엔진 설치
6. `~/.hermes/` 설정 디렉터리·SOUL.md 생성
7. `hermes` 명령어 심볼릭 링크 생성
8. `hermes setup` 대화형 설정 마법사 실행(`--skip-setup` 으로 건너뛸 수 있음)

PyPI 패키지 설치

v0.14.0부터 Hermes Agent를 PyPI 정식 패키지로 설치할 수 있습니다. 셸 스크립트를 거치지 않고 한 줄로 설치할 수 있습니다.

```
pip install hermes-agent
hermes
```

휠 안에 Ink TUI 번들과 셸 런처가 들어 있어 별도 빌드 없이 바로 실행할 수 있습니다. 독립된 도구 환경에 설치하고 싶다면 `uv tool install hermes-agent` 도 권장합니다.

원라인 셸 인스톨러는 Python·Node·ripgrep·ffmpeg를 함께 설치하므로 새 환경에 적합합니다. PyPI 설치는 이미 Python 환경이 있는 시스템에 Hermes만 추가할 때 적합합니다.

수동 설치

자동 스크립트 대신 단계별로 직접 설치할 수도 있습니다.

```
# 1. 저장소 클론 (서브모듈 포함)
git clone --recurse-submodules https://github.com/NousResearch/hermes-agent.git
cd hermes-agent

# 2. uv 설치 (Python 패키지 매니저)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 3. Python 3.11 가상환경 생성
uv venv --python 3.11

# 4. Python 의존성 설치 (lockfile 기반)
uv sync --extra all --locked

# 5. 설정 디렉터리 생성
mkdir -p ~/.hermes/{cron,sessions,logs,pairing,hooks,image_cache,audio_cache,memories,skills}

# 6. 예제 설정 복사
cp cli-config.yaml.example ~/.hermes/config.yaml

# 7. API 키 설정
cp .env.example ~/.hermes/.env
# ~/.hermes/.env를 편집하여 API 키 입력

# 8. 심볼릭 링크 (선택)
ln -sf $(pwd)/hermes ~/.local/bin/hermes

# 9. LLM 프로바이더 설정
hermes model
```

`uv sync --extra all`은 `[all]` extra에 들어 있는 의존성만 설치합니다. 메시징 어댑터, 음성·TTS, 이미지·비디오 SDK 같은 무거운 의존성은 v0.14의 경량화(디블로팅) 정책에 따라 `[all]`에 포함하지 않았습니다. 해당 기능은 처음 사용할 때 자동으로 설치됩니다. 자동 설치를 끈 환경에서는 직접 설치해야 합니다(「선택 기능 설치」 참고).

플랫폼별 설치와 주의사항

macOS

macOS에서는 **Xcode Command Line Tools**가 필요합니다.

```
xcode-select --install
```

또는 GUI 없이:

```
touch /tmp/.com.apple.dt.CommandLineTools.installondemand.in-progress
softwareupdate -l | grep "Command Line Tools"
softwareupdate -i "Command Line Tools for Xcode-13.2"   # 버전은 출력에 맞게 조정
```

Big Sur 이하 — Tier 3 우회

Big Sur(11.x) 이하에서는 Homebrew가 사전 빌드 바이너리(bottle)를 제공하지 않습니다(Tier 3 정책). install.sh가 설치하려는 ripgrep·ffmpeg 의존성을 모두 소스에서 빌드해야 해서 8GB RAM 머신에서는 끝까지 진행하기 어렵습니다. 대안은 다음과 같습니다.

1. **macOS 업그레이드** — Monterey 이상이 공식 지원되는 머신이라면 먼저 업그레이드합니다. Homebrew가 다시 bottle을 사용할 수 있어 설치가 몇 분 안에 끝납니다.
2. **MacPorts** — Big Sur 공식 pkg(www.macports.org)로 ripgrep·ffmpeg를 별도 설치하면 install.sh가 brew 단계를 건너뜁니다.
3. **수동 설치(대체 절차)** — ripgrep·ffmpeg 없이도 본체는 작동합니다. 음성 모드를 안 쓰는 첫 실습에는 영향이 없으니 둘 다 생략하고 위 「수동 설치」 절차로 진행해도 됩니다.
4. **장시간 빌드는 `screen` / `tmux` 로** — SSH 세션이 끊겨도 빌드가 살아남도록 합니다.

이 책의 실습 환경(12-inch MacBook 2015)도 Big Sur가 최종 지원이라 위 대체 절차를 사용했습니다.

Native Windows (베타)

v0.14.0에서 WSL 없이 `cmd.exe` 와 PowerShell에서 직접 실행하는 Native Windows 설치 방식이 베타로 추가됐습니다. PowerShell 인스톨러는 MinGit 자동 설치, Microsoft Store python stub 감지, 포그라운드 Ctrl+C 처리까지 자동으로 수행합니다. 아직 다듬어야 할 부분은 남아 있지만, 설치부터 실행까지 핵심 흐름은 새 Windows 환경에서 처음부터 끝까지 작동합니다. 정식 설치 명령은 PowerShell 인스톨러가 안정화된 뒤 본 절에 추가할 예정입니다.

Termux / Android

원라인 설치 스크립트는 Android의 Termux 환경을 자동으로 감지합니다(`TERMUX_VERSION` 또는 `PREFIX` 경로로 판별). 그에 맞게 패키지 매니저와 경로가 조정되므로, 일반 Linux와 같은 명령으로 설치하면 됩니다.

```
# Termux에서
pkg update && pkg install curl
curl -fsSL https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh
| bash
```

설치 후에는 모바일 화면에 맞춰 TUI가 작동하고, 음성 모드와 `/image` 명령도 사용할 수 있습니다.

> 💡 ***Termux 최적화**: 최근 버전에서 Termux 콜드 스타트 속도가 개선되었고, Python 3.13+에서 `sys.platform` 이 `"android"` 로 보고되어 Linux 태그 스킬(github-pr-workflow, google-workspace 등 약 60개)이 누락되던 문제도 수정되었습니다. 스킬 목록이 비정상적으로 적게 나오면 최신 버전으로 업데이트하십시오.*

일반 Linux / Ubuntu 데스크톱 설치 (비root)

일반 사용자(비root)로 원라인 설치를 실행하면 스크립트가 시스템을 건드리지 않고 필요한 런타임을 **Hermes 전용 영역 (`~/.hermes/`)에 따로 갖춥니다**. Ubuntu 데스크톱 기준으로 실제 흐름은 다음과 같습니다.

- **uv는 시스템이 아니라 `~/.hermes/bin` 에 설치됩니다.** 시스템 패키지 매니저를 거치지 않으므로 기존 Python 환경과 충돌하지 않습니다. 위 「수동 설치」는 표준 경로의 uv를 안내하지만, 원라인 설치는 이렇게 관리형 uv를 씁니다.
- **Python 3.11이 없으면 uv가 받아옵니다.** 시스템 Python을 바꾸지 않습니다.
- **시스템 Node.js가 너무 낡으면 관리형 Node를 따로 설치합니다.** 데스크톱·브라우저 빌드는 Node `^20.19 || ⩾22.12` 를 요구하는데, 이 조건을 못 채우면 Hermes가 Node 22 LTS를 `~/.hermes/node/` 에 받아 그 안에서만 씁니다. Ubuntu 22.04의 기본 Node 18이 여기에 해당하므로, 관리형 Node가 자동으로 설치되는 것이 정상입니다.
- **ripgrep·ffmpeg는 선택 항목입니다.** apt로 설치하므로 sudo 동의를 묻고, 거절할 수 있습니다(Hermes 본체는 root 권한이 필요 없습니다). ripgrep을 설치하지 않으면 파일 검색이 grep으로 대체되고, ffmpeg가 없으면 음성(TTS) 기능이 제한됩니다.
- **빌드 도구도 선택적으로 묻습니다.** 일부 Python 패키지를 빌드하는 데 필요한 `build-essential` · `python3-dev` · `libffi-dev` 가 없으면 설치 여부를 묻습니다(sudo). 휠(wheel) 설치만으로 충분한 경우가 많아 거절해도 대개 그대로 진행됩니다.
- **시스템 Chrome/Chromium이 있으면 Playwright 브라우저를 따로 받지 않습니다.** `/usr/bin/google-chrome` 같은 시스템 브라우저를 감지하면 다운로드를 생략하고, 그 경로를 `~/.hermes/.env` 의 `AGENT_BROWSER_EXECUTABLE_PATH` 에 기록해 재사용합니다.
- **클론은 SSH를 먼저 시도하고 실패하면 HTTPS로 넘어갑니다.** 로그에 `SSH failed, trying HTTPS ...` 가 보여도 오류가 아니라 정상적인 폴백입니다.

설치 위치는 다음과 같습니다.

항목	경로
코드	`~/.hermes/hermes-agent/`
데이터·설정	`~/.hermes/` (config.yaml·.env·SOUL.md·cron·sessions·logs 등)
관리형 런타임	`~/.hermes/bin/` (uv), `~/.hermes/node/` (Node)
명령어	`~/.local/bin/hermes`

> 💡 ***대화형 선택 프롬프트 확인**: 설치 스크립트의 선택 프롬프트(ripgrep·ffmpeg·빌드 도구 설치 여부 등)는 답하고 나면 화면에서 지워져, 나중에 터미널 기록을 봐도 무엇을 골랐는지 확인하기 어렵습니다. 무엇이 설치됐는지 다시 확인하려면 `hermes doctor` 를 실행하십시오.*

Linux root 설치와 FHS 레이아웃

Linux에서 root 사용자로 설치 스크립트를 실행하면 FHS(Filesystem Hierarchy Standard) 레이아웃이 적용됩니다.

항목	경로
코드	`/usr/local/lib/hermes-agent/`
명령어	`/usr/local/bin/hermes`
데이터	`$HERMES_HOME` (기본 `~/.hermes/`)

기존에 `~/.hermes/hermes-agent/` 에 설치된 환경이 있으면 레거시 레이아웃을 유지합니다. Docker 컨테이너에서 시스템 전역 설치를 할 때 특히 유용합니다.

Zed 에디터 연동

Zed 사용자는 Hermes를 설치한 뒤 Agent Client Protocol 레지스트리에 등록해 Zed 안에서 Hermes를 호출할 수 있습니다. 등록 경로가 `uvx` 라 npm 의존성이 필요 없습니다.

```
hermes acp --setup-browser   # 레지스트리 설치용 브라우저 도구 부트스트랩
```

본체(원라인 또는 PyPI)를 먼저 설치해야 합니다.

설치 확인과 초기 설정

설치가 끝났으면 셸을 다시 로드합니다.

```
source ~/.bashrc  # 또는 ~/.zshrc
```

`hermes doctor`로 Python 버전, 의존성, 설정 파일, API 키 등을 점검합니다.

```
hermes doctor
```

원라인 설치를 `--skip-setup`으로 건너뛰었거나 수동 설치를 마쳤다면 다음 명령으로 마법사를 다시 실행해 프로바이더·터미널 백엔드·메시징 플랫폼을 대화형으로 설정합니다.

```
hermes setup
```

이후 모델·프로바이더만 별도로 바꾸려면 `hermes model`을 씁니다. 자세한 모델 선택 절차는「클라우드 API 연동」절을 참고합니다.

선택 기능 설치 (lazy-install)

Hermes Agent는 메시징 어댑터, 음성·TTS 프로바이더, 이미지·비디오 생성 SDK 같은 무거운 선택 의존성을 **처음 사용할 때 자동으로 설치**합니다. 새로 설치한 직후에는 이들 SDK를 미리 받아오지 않으므로 디스크와 첫 실행이 가볍습니다.

```
> 텔레그램 게이트웨이를 설정해 줘
🔧 Installing python-telegram-bot … done (3.2s)
```

자동 설치가 보안 정책에 맞지 않는 환경에서는 `config.yaml`에서 비활성화할 수 있습니다.

```
security:
  allow_lazy_installs: false   # 런타임 자동 설치 차단
```

비활성화한 경우에는 필요한 기능을 직접 설치합니다.

```
# 메시징 게이트웨이
uv pip install -e ".[messaging]"

# 음성 모드
uv pip install -e ".[voice]"
```

> 💡 *공급망 점검(supply-chain advisory checker): `hermes doctor` 와 CLI 시작 시 알려진 악성 패키지(예: `mistralai 2.4.6`)가 설치돼 있으면 경고를 표시합니다. `hermes doctor --ack <id>` 로 확인 처리할 수 있습니다.*

기존 환경 이전 — `hermes backup` / `hermes import`

설정·세션·스킬·메모리를 통째로 백업해 다른 머신에서 복원할 수 있습니다.

```
hermes backup                # 전체 백업 (~/hermes-backup-<timestamp>.zip)
hermes backup -o my-backup.zip
hermes backup --quick -l "before-experiment"   # 핵심 상태 파일만 빠르게

# 다른 머신에서
hermes import my-backup.zip            # 기본은 비파괴(이미 있는 파일은 보존)
hermes import my-backup.zip --force    # 충돌 시 백업의 내용으로 덮어쓰기
```

`hermes update` 와 결합한 자동 백업 옵션은「업데이트」절을 참고합니다.

버전별 변경 이력

- **v0.14.0** — PyPI 정식 배포(`pip install hermes-agent`), Native Windows 베타, Zed ACP Registry 통합. 지연 설치(lazy-install) 대상 확장(Feishu·DingTalk 어댑터, 음성/TTS 프로바이더, hindsight 클라이언트, codex app-server, Pixverse·Camofox SDK가 추가로 `[all]` 에서 빠짐). 단계별 설치(tiered install) 대체 절차 도입.
- **v0.13.0** — 지연 설치(lazy-install) 도입(메시징 어댑터·Edge TTS·ElevenLabs·Bedrock·Honcho·FAL 등이 `[all]` 에서 빠짐). Linux root 설치 시 FHS 레이아웃 적용.

02-3. 클라우드 API 연동

- 프로바이더 결정 방식
- 클라우드 API
  - Anthropic (Claude)
  - OpenAI (GPT)
  - OpenRouter (기본 프로바이더)
  - 기타 정식 프로바이더
- hermes model — 프로바이더와 모델 바꾸기
- hermes proxy — OAuth 프로바이더용 로컬 프락시 (v0.14+)
- 폴백 프로바이더
- Hugging Face
- 연동 확인
- 보조 모델 설정
  - 보조 작업 종류
  - 기본 작동
  - 설정 예시

이 절은 클라우드 API와 연동하는 방법을 다룹니다. 로컬 LLM(Ollama·LM Studio 등) 설정은 부록 「로컬 LLM 가이드」에서 자세히 다룹니다.

> 💡 ***빠른 시작**: 프로바이더 메커니즘을 건너뛰고 바로 연동하고 싶다면 다음 절 「클라우드 모델 연결하기」을 먼저 따라 해도 됩니다.*

프로바이더 결정 방식

요청이 어디로 가는지는 **모델명이 아니라 프로바이더 설정**이 결정합니다. 모델명의 슬래시 접두사(`anthropic/`, `openai/`)는 OpenRouter 등에서 쓰는 모델 식별자일 뿐, 라우팅에 관여하지 않습니다.

프로바이더 결정 우선순위:

1. CLI `--provider` 플래그
2. `config.yaml`의 `model.provider`
3. 환경 변수 `HERMES_INFERENCE_PROVIDER`
4. 위 셋 모두 없으면 **"auto"** — 시작 시 사용 가능한 자격 증명을 자동 탐지합니다

엔드포인트 URL의 단일 출처는 **`config.yaml`의 `model.base_url`** 입니다. v0.7.0 이전에는 `.env`의 `OPENAI_BASE_URL`도 함께 참조됐지만, 그 이후로는 더 이상 쓰지 않습니다(`runtime_provider.py` 주석으로도 명시).

> API 키만 .env에 추가해서는 프로바이더가 바뀌지 않습니다
>
> 초보자가 가장 자주 빠지는 함정입니다. 예를 들어 이미 로컬 Ollama로 작동하도록 `model.provider: custom` · `model.base_url: http://…:11434/v1`이 `config.yaml`에 잡혀 있을 때, 단지 `.env`에 `OPENROUTER_API_KEY=…`를 추가해도 라우팅은 그대로 Ollama입니다.
>
> `.env`의 API 키는 **이미 결정된 base_url에 어떤 자격 증명을 붙일지를 고르는 역할**일 뿐, 어떤 프로바이더로 갈지는 결정하지 않습니다. 프로바이더를 바꾸려면 다음 중 하나를 합니다.
>
> - `hermes model`로 새 프로바이더와 모델 선택(아래 "실전" 절 참고) — 가장 권장
> - `config.yaml`의 `model.provider` / `model.base_url` / `model.default`를 직접 수정
> - 일회성 사용이면 `hermes chat --provider <slug> -m <model>` 또는 `hermes -z --provider … -m …`

클라우드 API

클라우드 API는 설치가 간편하고 모델 성능이 높습니다. 데이터가 외부로 전송되며 사용량에 따라 비용이 발생합니다.

Anthropic (Claude)

```
# ~/.hermes/.env
ANTHROPIC_API_KEY=sk-ant-…
```

Claude를 주 모델로 사용하려면 `config.yaml`에서 모델과 프로바이더를 지정합니다.

```
model:
  default: "anthropic/claude-sonnet-4-6"
  provider: "anthropic"
```

또는 CLI에서 직접 지정할 수도 있습니다.

```
hermes chat --provider anthropic -m anthropic/claude-sonnet-4-6
```

`provider: "anthropic"`을 명시하면 Anthropic API로 직접 요청이 갑니다. 모델명의 `anthropic/` 접두사는 모델 식별자이며, 라우팅을 결정하는 것은 `provider` 설정입니다.

현재 사용 가능한 주요 모델:

모델	API ID	입력 / 출력 (MTok당)	특징
Claude Sonnet 4.6	`claude-sonnet-4-6`	\$3 / \$15	속도와 성능의 균형, 일반 용도 권장
Claude Opus 4.8	`claude-opus-4-8`	\$5 / \$25	최신 최고 성능, 복잡한 에이전트 작업
Claude Haiku 4.5	`claude-haiku-4-5`	\$1 / \$5	가장 빠르고 저렴

OpenAI (GPT)

```
# ~/.hermes/.env
OPENAI_API_KEY=sk- ...
```

```
# ~/.hermes/config.yaml
model:
  default: "openai/gpt-4o"
  provider: "openai"
```

OpenRouter (기본 프로바이더)

OpenRouter는 Hermes Agent의 **기본 프로바이더**입니다. 별도의 `provider`를 지정하지 않으면 모든 요청은 OpenRouter를 통해 전달됩니다. 하나의 API 키로 Claude, GPT, Gemini, Llama 등 다양한 모델에 접근할 수 있습니다.

```
# ~/.hermes/.env
OPENROUTER_API_KEY=sk-or- ...
```

별도의 `provider` 설정 없이 모델명만 지정하면 됩니다.

```
# ~/.hermes/config.yaml
model:
  default: "anthropic/claude-sonnet-4-6"
```

Anthropic이나 OpenAI에 직접 연결하려면 해당 `provider`를 명시해야 합니다. `provider`가 없으면 OpenRouter 경유입니다.

OpenRouter 카탈로그에는 `:free` 접미사가 붙은 무료 모델이 종종 등장합니다(예: `nvidia/nemotron-3-super-120b-a12b:free`). 결제 수단·크레딧이 부족할 때 시험 삼아 띄워 보기 좋습니다. 단 무료 모델은 속도 제한·가용성이 들쭉날쭉할 수 있고, **프롬프트와 응답이 프로바이더의 모델·서비스 개선 목적으로 로깅**됩니다(모델별 페이지의 공식 고지 참고). 개인정보·기밀 정보·업무 데이터는 입력하지 말고, 운영 환경에서는 유료 모델이나 로컬 LLM(부록 「로컬 LLM 가이드」)을 사용하세요.

기타 정식 프로바이더

Anthropic·OpenAI·OpenRouter 외에도 Hermes Agent에는 다양한 추론 경로가 정식 프로바이더로 등록되어 있습니다. 각 프로바이더는 고유 `provider` 슬러그와 환경 변수를 가지며, `hermes model` 명령(대화 중에는 `/model`)에서 골라 쓸 수 있습니다. 한국에서 자주 쓰는 후보 위주로 정리하면 다음과 같습니다.

클라우드 / 빅테크 (자체 키)

프로바이더	`provider`	비고
Google AI Studio (Gemini)	`google`	Gemini 모델을 AI Studio API로 직접 호출
AWS Bedrock	`bedrock`	Converse API 기반 네이티브 지원, AWS SDK 자격증명 사용
Azure AI Foundry	`azure-foundry`	OpenAI/Anthropic 양쪽 모드를 `model.api_mode`로 선택
xAI (Grok, API 키)	`xai`	Codex Responses API로 직접 호출

OAuth 로그인 기반 (브라우저 로그인 후 키 없이 사용)

프로바이더	`provider`	비고
Nous Portal	`nous`	NousResearch 공식 포털, OAuth device code
ChatGPT Codex (GPT-5.5 등)	`codex`	ChatGPT Codex 구독으로 GPT-5.5 reasoning 모델 접근
GitHub Copilot	`copilot`	GitHub Copilot 구독 활용. `copilot-acp` (외부 프로세스 인증)도 별도 지원
xAI SuperGrok OAuth	`xai-supergrok`	v0.14 신규 — SuperGrok 구독으로 xAI 계정 로그인. **grok-4.3는 1M 컨텍스트 윈도우**로 확장됨. SSH 환경에서 OAuth를 완료하기 위한 터널 안내 문서 포함. 2026-05-15 일부 Grok 모델(`grok-3` , `grok-4-0709` , `grok-4-fast-*` 등) 폐기 — 기존 config가 폐기 모델을 참조하면 `hermes migrate xai` 로 진단·치환(자세한 절차는 「2-4 프로바이더 빠른 시작」 절)
Google Gemini CLI OAuth	`google-gemini-cli`	Code Assist 채널
Qwen OAuth	`qwen-oauth`	알리바바 Qwen 포털 OAuth
MiniMax OAuth	`minimax-oauth`	PKCE 브라우저 OAuth

중국 모델 직접 연결

프로바이더	`provider`	비고
알리바바 Qwen	`alibaba`	DashScope 엔드포인트
DeepSeek	`deepseek`	DeepSeek v4 시리즈
Moonshot Kimi	`kimi-coding`	Kimi 코딩 플랜. 중국 엔드포인트는 `kimi-coding-cn`
샤오미 MiMo	`xiaomi`	MiMo v2 Pro 등
MiniMax (API 키)	`minimax`	Anthropic Messages 호환. 중국 엔드포인트는 `minimax-cn`
ZAI (GLM)	`zai`	智谱 GLM
Tencent Tokenhub	`tencent-tokenhub`	텐센트 클라우드 모델 게이트웨이
StepFun (Step Plan)	`stepfun`	StepPlan 모델
알리바바 Coding Plan	`alibaba-coding-plan`	Alibaba Cloud 코딩 전용 플랜

오픈소스/스타트업 게이트웨이

프로바이더	`provider`	비고
Vercel AI Gateway	`vercel`	가격·동적 모델 발견 포함
KiloCode	`kilo`	애그리게이터
OpenCode Zen	`opencode`	35+ 큐레이션 모델, 종량제. `opencode-go` (월 \$10 구독)도 지원
Hugging Face	`huggingface`	Inference API 애그리게이터
GMI Cloud	`gmi`	정식 API 키 프로바이더
Arcee AI	`arcee`	Arcee 모델 직접
NVIDIA NIM	`nvidia`	NVIDIA build API
NovitaAI	`novita`	v0.14 신규 — Llama·Qwen·DeepSeek 등 오픈소스 모델 호스팅. 자체 단가와 속도 제한
LM Studio	`lmstudio`	로컬 LM Studio 서버 연결
Ollama Cloud	`ollama-cloud`	Ollama 클라우드 호스팅

> 💡 *위 표는 v0.13.0 기준 정식 프로바이더입니다. 모델 카탈로그(어떤 모델 ID가 사용 가능한지)는 OpenRouter·Nous Portal에 한해 **원격 매니페스트로 자동 갱신**되므로, 새 모델이 등장해도 Hermes를 업데이트하지 않고 `hermes model` 에서 바로 보입니다. 그 외 프로바이더는 모델 카탈로그가 코드에 묶여 있어 새 모델이 안 보이면 업데이트가 필요합니다.*

로컬 추론 경로(LM Studio·Ollama·vLLM 등)는 부록「로컬 LLM 가이드」에서 별도로 다룹니다.

`hermes model` — 프로바이더와 모델 바꾸기

설정 파일을 직접 손대지 않고 인터랙티브 피커로 프로바이더와 모델을 바꿀 수 있습니다. 처음 설정할 때나 다른 프로바이더로 옮길 때 권장하는 방법입니다.

```
hermes model
```

선택을 마치면 `config.yaml`의 `model.default`, `model.provider`, `model.base_url`, `model.api_mode`가 자동으로 갱신됩니다. 대화 중에는 슬래시 명령 `/model`로도 같은 피커를 띄울 수 있습니다(3-3절 참고). 화면 예시와 단계별 절차는 다음 절「클라우드 모델 연결하기」에서 다룹니다.

`hermes proxy` — OAuth 프로바이더용 로컬 프락시 (v0.14+)

OAuth 로그인만 제공하는 프로바이더를 Aider, Open WebUI 같은 외부 도구에서 쓸 수 있게 해 주는 OpenAI 호환 로컬 프락시입니다. v0.14.0에는 Nous Portal 어댑터만 포함되어 있습니다. 자세한 사용법은 12장의 `hermes proxy` 절을 참고하세요.

폴백 프로바이더

주 프로바이더가 실패할 때 자동으로 다른 프로바이더에 재시도하는 체인을 설정할 수 있습니다. 2-5절의 `fallback_providers` 항목을 참고하세요. 명령줄에서 빠르게 점검·관리하려면 `hermes fallback` 명령을 씁니다.

```
hermes fallback list      # 등록된 폴백 체인 보기
hermes fallback add       # 새 폴백 프로바이더 추가
hermes fallback remove    # 폴백 프로바이더 삭제
hermes fallback clear     # 전체 폴백 체인 초기화
```

Hugging Face

Hugging Face를 추론 제공자로 직접 사용할 수 있습니다. Hugging Face의 Inference API를 통해 다양한 오픈소스 모델에 접근할 수 있습니다.

```
# config.yaml
model:
  default: "meta-llama/Llama-3.1-8B-Instruct"
  provider: "huggingface"
```

```
# .env
HF_TOKEN=hf_xxxxxxxx
```

연동 확인

설정이 끝나면 연동을 확인합니다.

```
hermes chat -q "안녕, 너는 어떤 모델이야?"
```

정상적으로 연결되면 배너에 모델 이름이 표시되고, 응답이 출력됩니다. 연결에 문제가 있다면 `hermes doctor`로 진단할 수 있습니다.

> 💡 *여러 프로바이더의 키를 `.env`에 함께 넣어 두고, `hermes model`(또는 일회성 `hermes chat --provider`)로 사이를 오갈 수 있습니다. `.env`에는 각 프로바이더의 API 키만 넣고, 엔드포인트와 모델 설정은 모두 `config.yaml`에서 관리합니다.*

보조 모델 설정

Hermes Agent는 주 모델과 별도로 보조 모델(auxiliary models)을 설정할 수 있습니다. 보조 모델은 주 모델보다 가볍고 빠른 모델을 배정하여 비용과 속도를 최적화하는 데 사용합니다.

보조 작업 종류

작업	용도	기본 타임아웃
`vision`	이미지/스크린샷 분석	120초
`web_extract`	웹페이지 요약	360초
`compression`	컨텍스트 압축 (대화 요약)	120초
`session_search`	세션 검색	30초
`skills_hub`	스킬 허브 검색	30초
`approval`	위험 명령 자동 판단	30초
`mcp`	MCP 도구 호출 보조	30초
`title_generation`	세션 제목 자동 생성	30초
`curator`	백그라운드 스킬 큐레이터(7장 참고)	600초

기본 작동

모든 보조 작업의 기본 provider는 `"auto"` 이고 model은 빈 문자열입니다. 이 경우 OpenRouter를 통해 `google/gemini-3-flash-preview` 로 폴백됩니다. 별도 설정 없이도 작동하지만, 로컬 모델을 지정하면 비용을 절감할 수 있습니다.

설정 예시

클라우드 보조 모델로 통일하는 구성:

```
auxiliary:
  vision:
    provider: "auto"       # 기본값 유지 (Gemini Flash 폴백)
    model: ""
  web_extract:
    provider: "auto"
    model: ""
  compression:
    provider: "auto"
    model: ""
```

로컬 모델을 보조로 쓰려면(부록 참고): `provider: "custom"` + 로컬 서버 `base_url`로 슬롯을 따로 채우면 됩니다. vision·web_extract처럼 부수 작업만 로컬로 돌리고, 메인 모델은 클라우드를 쓰는 혼합 구성도 가능합니다.

02-4. [빠른 시작] 클라우드 모델 연결하기

Hermes Agent는 다양한 클라우드 LLM 프로바이더를 지원합니다. 이 가이드는 최소한의 설정으로 에이전트의 기능을 온전히 확보할 수 있도록, 기본 대화를 위한 **OpenRouter**와 멀티모달 도구 확장을 위한 **xAI SuperGrok**을 연결하는 가장 빠른 경로를 안내합니다. 전체 프로바이더 사양은 「클라우드 API 연동」 절을 참고하세요.

- 시작하기 전에
- OpenRouter 연동
  - 1단계 — API 키 발급
  - 2단계 — API 키 등록
  - 3단계 — 프로바이더와 모델 선택
  - 4단계 — 연동 확인
- xAI SuperGrok 연동 — 도구 확장
  - 1단계 — 인증 등록
  - 2단계 — 도구 켜기
  - 3단계 — 인증 확인
  - Grok을 채팅 모델로도 쓰려면
  - SSH·원격 환경에서 로그인

시작하기 전에

> 💡 ***사전 준비**: Hermes Agent가 설치되어 있어야 합니다. 아직 설치하지 않았다면 「설치하기」 절을 먼저 참고하세요.*

두 프로바이더는 에이전트 작동에 필요한 필수 대화 레이어(채팅)와 선택적 기능 레이어(도구)를 각각 분담합니다.

- **OpenRouter** — 채팅 모델용 기본 프로바이더입니다. 무료 모델을 제공하므로 결제 없이 바로 시작할 수 있습니다.
- **xAI SuperGrok** — TTS·이미지 생성·비디오 생성·X 검색 도구를 사용하기 위해 인증을 등록합니다. 월정액 구독이라 비용 상한이 정해져 있고, 브라우저에서 로그인하기만 하면 연결할 수 있습니다.

OpenRouter 하나만 연결해도 Hermes Agent의 기본 기능은 모두 사용할 수 있으므로, 먼저 OpenRouter를 설정하고, 필요할 때 SuperGrok을 추가하세요.

OpenRouter 연동

OpenRouter는 Hermes Agent의 기본 프로바이더입니다. API 키 하나로 Claude, GPT, Gemini, Llama 등 여러 모델을 사용할 수 있어 처음 시작할 때 가장 편합니다.

1단계 — API 키 발급

openrouter.ai에 가입하고, 대시보드의 Keys 메뉴에서 API 키를 발급받습니다.

2단계 — API 키 등록

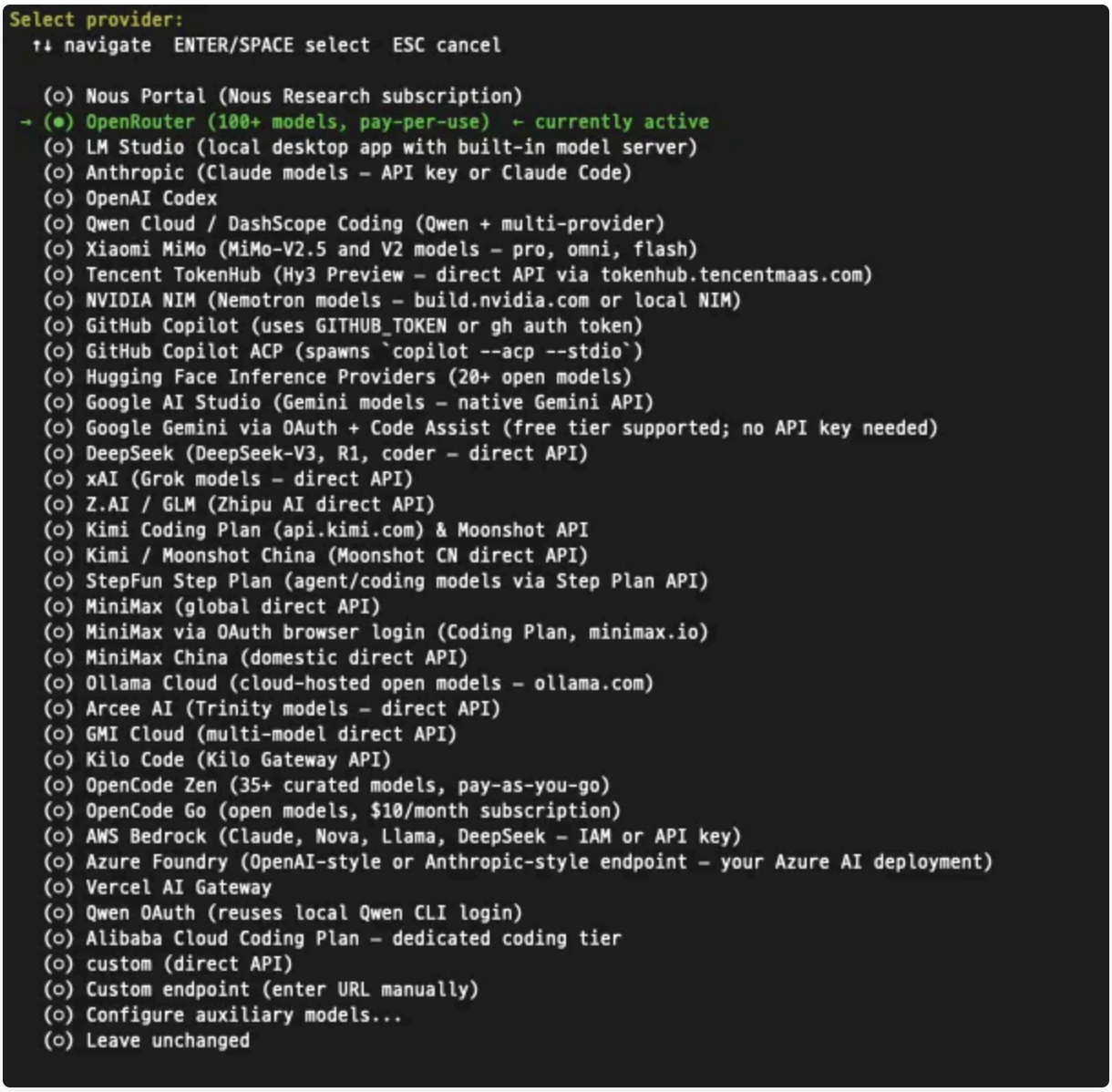
발급받은 키를 `~/.hermes/.env` 파일에 추가합니다. `~/.hermes/`는 Hermes Agent의 설정 디렉터리이며, 설치하면 자동으로 생깁니다. `.env` 파일이 없으면 새로 만듭니다.

```
OPENROUTER_API_KEY=sk-or- ...
```

3단계 — 프로바이더와 모델 선택

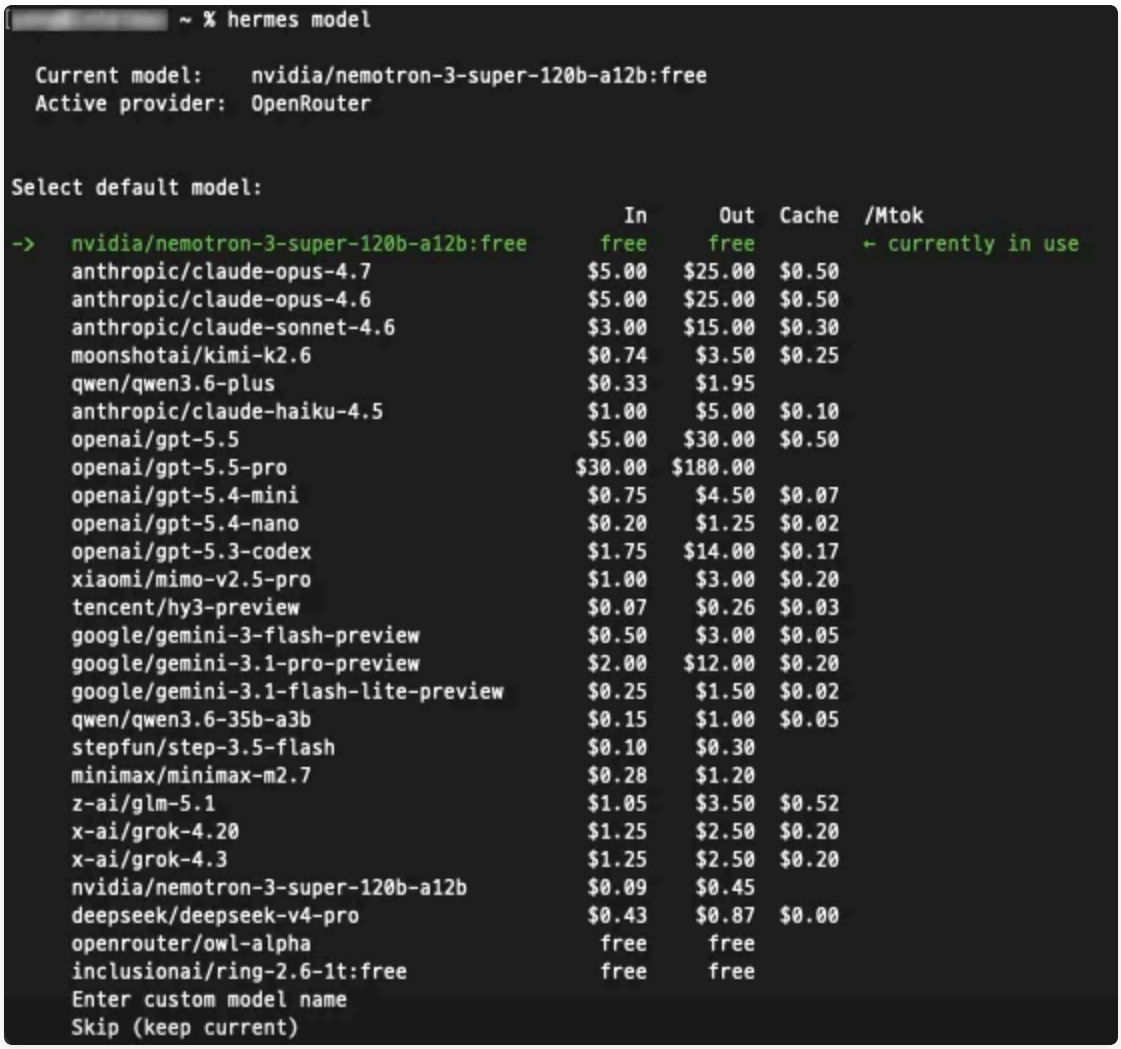
```
hermes model
```

명령을 실행하면 프로바이더 목록이 열립니다. 위·아래 방향키로 이동하고 Enter 키로 선택합니다.



목록 위쪽이 자주 쓰는 클라우드 프로바이더(OpenRouter·Anthropic·OpenAI Codex·Google AI Studio 등), 가운데가 OAuth 로그인 기반 프로바이더, 아래가 직접 키를 넣는 기타 프로바이더 순서입니다. `●`은 현재 선택된 프로바이더이고, 나머지는 `○`로 표시됩니다. **OpenRouter**를 선택합니다.

프로바이더를 고르면 모델 목록이 나옵니다. 입출력 단가도 함께 나오므로 비용을 비교하며 고를 수 있습니다.



`In` / `Out` / `Cache` 열이 모두 `free`인 항목이 무료 모델입니다(`:free` 접미사). 크레딧이 없을 때 작동을 먼저 확인하기 좋습니다.

> ⚠️ ***유료 모델 선택 시 크레딧 확인**: 카탈로그 맨 위의 인기 모델이 매력적으로 보이지만, 계정에 크레딧이 없으면 처음 호출할 때 결제 오류가 납니다. 무료 모델로 작동을 먼저 확인하고, 크레딧을 충전한 뒤 유료 모델로 바꾸는 것을 권합니다.*

무료 모델의 제한

무료 모델은 속도 제한과 가용성이 일정하지 않고, 서비스 개선을 위해 프롬프트와 응답이 기록될 수 있습니다. 개인정보나 업무 데이터는 입력하지 않는 것이 좋습니다.

선택을 마치면 Hermes가 `~/.hermes/config.yaml`의 `model.default`, `model.provider`, `model.base_url`, `model.api_mode` 값을 자동으로 갱신합니다. 대화 중에 모델을 바꾸고 싶다면 슬래시 명령 `/model`로도 같은 선택 창을 열 수 있습니다.

4단계 — 연동 확인

```
hermes chat -q "안녕, 너는 어떤 모델이야?"
```

배너에 모델 이름이 표시되고 응답이 돌아오면 연동에 성공한 것입니다.

`config.yaml`을 직접 편집할 때는 모델명만 지정하면 됩니다. OpenRouter가 기본 프로바이더이므로 `provider`를 생략해도 됩니다.

```
# ~/.hermes/config.yaml
model:
  default: "anthropic/claude-sonnet-4-6"
```

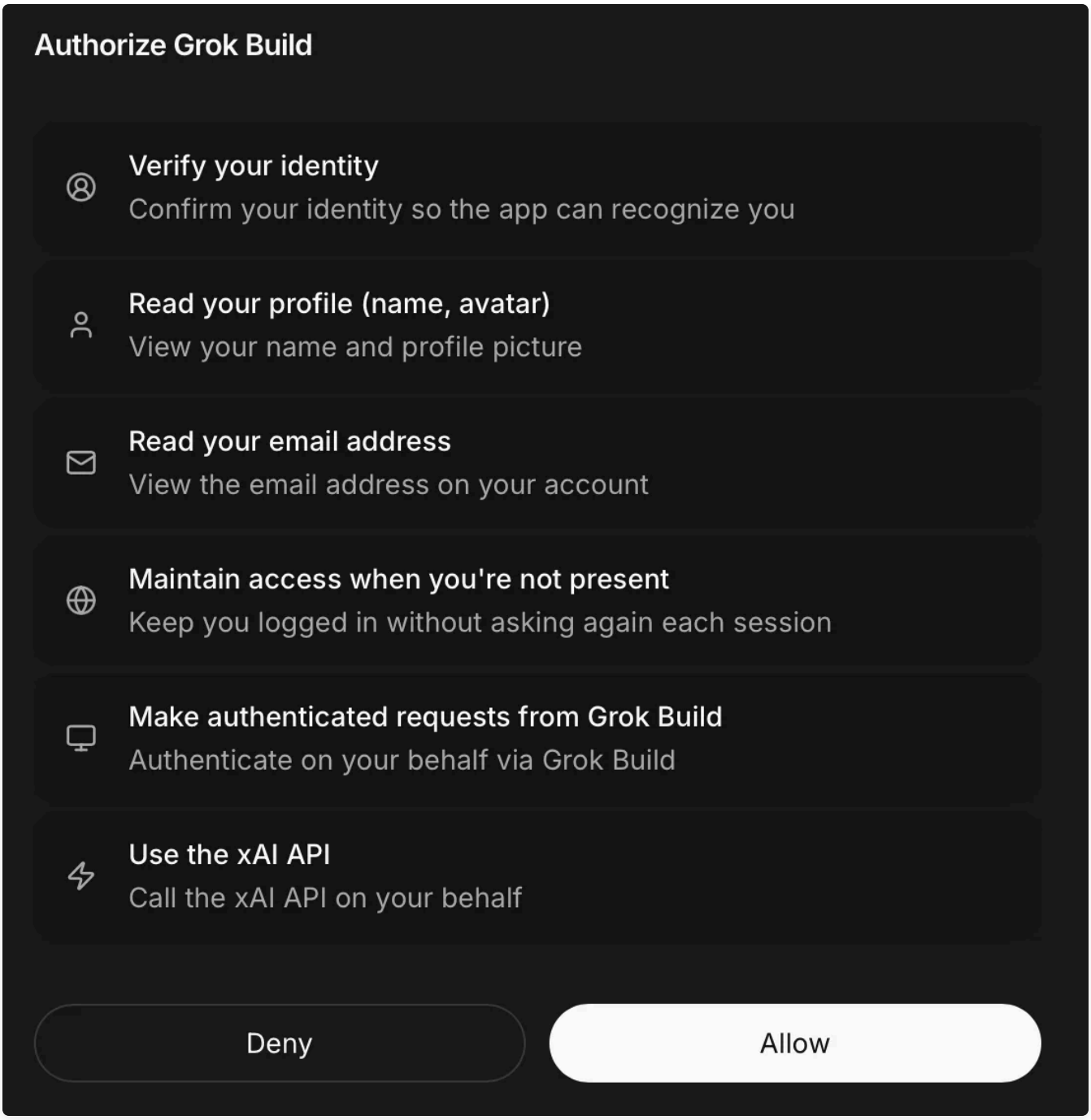
xAI SuperGrok 연동 — 도구 확장

SuperGrok 인증을 등록하면 앞서 설정한 OpenRouter 채팅 모델은 그대로 두고, TTS·이미지 생성·비디오 생성·X 검색 도구만 추가로 사용할 수 있습니다. **SuperGrok** 구독이 필요합니다(SuperGrok Lite는 OAuth를 지원하지 않습니다). 요금제는 x.ai/grok에서 확인할 수 있습니다.

1단계 — 인증 등록

```
hermes auth add xai-oauth
# → 브라우저가 열리며 accounts.x.ai 로그인 화면으로 이동
# → 로그인 후 접근 승인
```

로그인하면 "Authorize Grok Build" 화면이 나옵니다. **Allow**를 눌러 접근을 승인합니다.



Hermes는 인증 정보를 `~/.hermes/auth.json`에 저장하고, 토큰이 만료되기 전에 자동으로 갱신합니다. 이 단계에서는 채팅 모델이 바뀌지 않으므로, 앞서 설정한 OpenRouter 모델이 그대로 유지됩니다.

2단계 — 도구 켜기

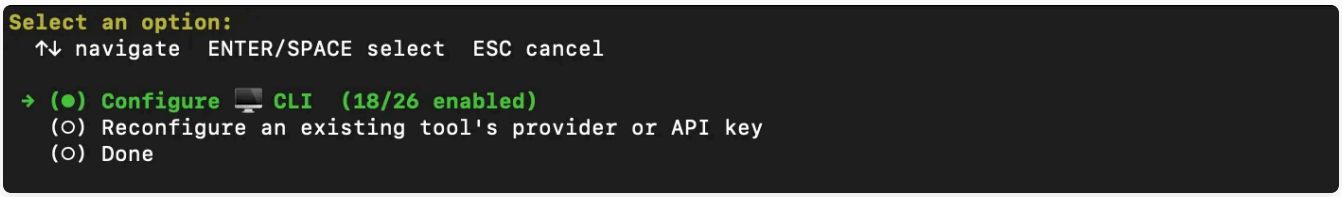
인증이 등록되면 `hermes tools`에서 SuperGrok 도구를 켤 수 있습니다.

도구	기본 상태	활성화 경로
TTS (음성 합성)	사용 가능	`hermes tools` → Text-to-Speech → xAI TTS
이미지 생성	사용 가능	`hermes tools` → Image Generation → xAI Grok Imagine
비디오 생성	**꺼져 있음**	`hermes tools` → Video Generation (스페이스바로 활성화)
X 검색	**꺼져 있음**	`hermes tools` → X (Twitter) Search (스페이스바로 활성화)

> ⚠️ ***기본으로 꺼진 도구**: 비디오 생성(`video_generate`)과 X 검색(`x_search`)은 기본적으로 꺼져 있습니다. 에이전트가 이 도구들을 사용하려면 `hermes tools` 에서 먼저 켜야 합니다.*

여기서는 X 검색을 켜는 과정을 예로 보여줍니다.

1. `hermes tools` 실행 후 **Configure CLI**를 선택합니다.



2. 도구 목록에서 X (Twitter) Search 항목으로 이동합니다. 기본 상태는 `[ ]` (꺼짐)입니다.

```
Tools for 🖥️ CLI
  ↑↓ navigate  SPACE toggle  ENTER confirm  ESC cancel

   [✓] 🔍 Web Search & Scraping  (web_search, web_extract)
   [✓] 🌐 Browser Automation  (navigate, click, type, scroll)
   [✓] 💻 Terminal & Processes  (terminal, process)
   [✓] 📁 File Operations  (read, write, patch, search)
   [✓]  Code Execution  (execute_code)
   [✓] 👁️ Vision / Image Analysis  (vision_analyze)
   [ ] 🎬 Video Analysis  (video_analyze (requires video-capable model))
   [✓] 🎨 Image Generation  (image_generate)
   [ ] 🎬 Video Generation  (video_generate (text-to-video + image-to-video))
 → [ ] 🐦 X (Twitter) Search  (x_search (requires xAI OAuth or XAI_API_KEY))
   [ ]  Mixture of Agents  (mixture_of_agents)
   [✓] 🔊 Text-to-Speech  (text_to_speech)
   [✓] 📚 Skills  (list, view, manage)
   [✓] 📋 Task Planning  (todo)
   [✓] 💾 Memory  (persistent memory across sessions)
   [✓] 🔎 Session Search  (search past conversations)
   [✓]  Clarifying Questions  (clarify)
   [✓] 👥 Task Delegation  (delegate_task)
   [✓] ⏰ Cron Jobs  (create/list/update/pause/resume/run, with optional attached skills)
   [✓] 📨 Cross-Platform Messaging  (send_message)
   [ ] 🏠 Home Assistant  (smart home device control)  [no API key]
   [ ] 🎵 Spotify  (playback, search, playlists, library)
   [ ] 🫧 Yuanbao  (group info, member queries, DM)
   [✓] 🖱️ Computer Use (macOS)  (background desktop control via cua-driver)
```

3. **스페이스바**를 눌러 `[✓]` (활성)로 바꿉니다. 원하는 도구를 모두 켠 뒤 **Enter** 키로 확인합니다.

```
Tools for 🖥️ CLI
  ↑↓ navigate  SPACE toggle  ENTER confirm  ESC cancel

   [✓] 🔍 Web Search & Scraping  (web_search, web_extract)
   [✓] 🌐 Browser Automation  (navigate, click, type, scroll)
   [✓] 💻 Terminal & Processes  (terminal, process)
   [✓] 📁 File Operations  (read, write, patch, search)
   [✓]  Code Execution  (execute_code)
   [✓] 👁️ Vision / Image Analysis  (vision_analyze)
   [ ] 🎬 Video Analysis  (video_analyze (requires video-capable model))
   [✓] 🎨 Image Generation  (image_generate)
   [ ] 🎬 Video Generation  (video_generate (text-to-video + image-to-video))
 → [✓] 🐦 X (Twitter) Search  (x_search (requires xAI OAuth or XAI_API_KEY))
   [ ]  Mixture of Agents  (mixture_of_agents)
   [✓] 🔊 Text-to-Speech  (text_to_speech)
   [✓] 📚 Skills  (list, view, manage)
   [✓] 📋 Task Planning  (todo)
   [✓] 💾 Memory  (persistent memory across sessions)
   [✓] 🔎 Session Search  (search past conversations)
   [✓]  Clarifying Questions  (clarify)
   [✓] 👥 Task Delegation  (delegate_task)
   [✓] ⏰ Cron Jobs  (create/list/update/pause/resume/run, with optional attached skills)
   [✓] 📨 Cross-Platform Messaging  (send_message)
   [ ] 🏠 Home Assistant  (smart home device control)  [no API key]
   [ ] 🎵 Spotify  (playback, search, playlists, library)
   [ ] 🫧 Yuanbao  (group info, member queries, DM)
   [✓] 🖱️ Computer Use (macOS)  (background desktop control via cua-driver)
```

4. 메인 화면으로 돌아오면 활성 도구 수가 늘어난 것을 확인할 수 있습니다. **Done**을 선택하고 Enter 키를 눌러 종료합니다.

```
Select an option:
  ↑↓ navigate  ENTER/SPACE select  ESC cancel

→ (●) Configure 🖥️ CLI  (19/26 enabled)
  (○) Reconfigure an existing tool's provider or API key
  (○) Done
```

3단계 — 인증 확인

`hermes doctor`의 `◆ Auth Providers` 섹션에서 xAI OAuth 인증 상태를 확인할 수 있습니다.

Grok을 채팅 모델로도 쓰려면

> 💡 ***선택사항**: 위의 1~3단계만 완료하면 OpenRouter 채팅 + SuperGrok 도구 구성이 끝납니다. 아래는 채팅 모델까지 Grok으로 바꾸고 싶을 때만 진행합니다.*

SuperGrok 인증을 등록했다면 채팅 모델도 Grok으로 바꿀 수 있습니다. `hermes model`에서 "xAI Grok OAuth (SuperGrok Subscription)"를 선택하고 모델을 고릅니다. 기본 모델은 `grok-4.3`이며 목록 맨 위에 고정되어 있습니다.

`hermes model` 대신 직접 설정할 수도 있습니다.

```
hermes config set model.default grok-4.3
hermes config set model.provider xai-oauth
```

프로바이더 별칭으로 `grok-oauth`, `x-ai-oauth`, `xai-grok-oauth`도 사용할 수 있습니다. 다시 OpenRouter로 돌아가려면 `hermes model`에서 OpenRouter를 선택하면 됩니다.

SSH·원격 환경에서 로그인

SSH 세션이나 컨테이너처럼 브라우저를 열 수 없는 환경에서는 Hermes가 인증 URL을 터미널에 출력합니다. OAuth 콜백 리스너가 원격 호스트의 `127.0.0.1:56121`에서 요청을 기다리므로, 로컬 머신에서 포트 포워딩을 설정해야 합니다.

```
# 로컬 머신의 별도 터미널에서
ssh -N -L 56121:127.0.0.1:56121 user@remote-host

# 원격 머신에서
hermes auth add xai-oauth --no-browser
# 출력된 URL을 로컬 브라우저에서 열어 로그인
```

점프 호스트를 거치는 환경이라면 `-J jump-user@jump-host` 옵션을 추가합니다.

02-5. 설정 파일 이해하기

- 디렉터리 구조
- 설정 우선순위
  - config.yaml에서 환경 변수 참조
- 설정 관리 명령어
- 모델과 라우팅
  - 모델 설정
  - OpenRouter 프로바이더 라우팅
  - 폴백 모델
  - 크리덴셜 풀
  - 보조 모델
  - 서브에이전트 위임 모델
  - Mixture of Agents (MoA)
- 그 밖의 주요 설정 항목
  - 터미널 백엔드
  - 메모리 설정
  - 컨텍스트 압축
  - 프롬프트 캐싱 (Anthropic)
  - 보안 — 시크릿 마스킹
  - 보안 — lazy-install 및 네트워크
  - 표시 및 입력
  - 에이전트 작동
- 마이그레이션 노트
  - v0.7.0 — OPENAI_BASE_URL 환경 변수 무효화
  - v0.13.0 — smart_model_routing 제거
- 설정 진단

Hermes Agent의 설정은 `~/.hermes/` 디렉터리에 저장됩니다. 이 절에서는 주요 설정 파일의 구조와 역할을 설명합니다.

디렉터리 구조

```
~/.hermes/
├── config.yaml        # 메인 설정 파일
├── .env               # API 키와 시크릿
├── auth.json          # OAuth 인증 정보 + 크리덴셜 풀 키 상태
├── SOUL.md            # 에이전트 성격 정의 (시스템 프롬프트)
├── memories/          # 영속 메모리 파일
├── skills/            # 에이전트가 생성한 스킬
├── cron/              # 예약 작업
├── sessions/          # 게이트웨이 세션
├── logs/              # 오류 및 게이트웨이 로그
├── pairing/           # DM 페어링 데이터
├── hooks/             # 도구 호출 훅
├── image_cache/       # 이미지 캐시
└── audio_cache/       # 음성 캐시
```

설정 우선순위

설정값의 적용 우선순위는 다음과 같습니다 (높은 것이 우선).

1. **CLI 인수**: 실행 시 지정한 옵션 (예: `hermes chat --model "..."`)
2. **config.yaml**: 메인 설정 파일
3. **.env**: 환경 변수 (주로 API 키)
4. **내장 기본값**: 코드에 하드코딩된 안전한 기본값

핵심 원칙: **시크릿(API 키, 봇 토큰, 비밀번호)은 `.env`에, 나머지는 모두 `config.yaml`에** 저장합니다.

이렇게 분리하는 데는 두 가지 이유가 있습니다. 첫째, `config.yaml`은 다른 머신과 공유하거나 dotfiles 저장소로 git에 올릴 가능성이 있는 설정 파일이고, `.env`는 거기서 시크릿만 떼어 두는 관례적인 곳(보통 `.gitignore`에 등록)입니다. 둘이 섞여 있으면 키가 함께 새 나갑니다. 둘째, `hermes doctor`·이슈 리포트·디버그 덤프 같은 도구가 `config.yaml`은 그대로 출력하는 반면 `.env`는 마스킹하거나 생략합니다. 1Password CLI 같은 시크릿 매니저도 보통 환경 변수로 키를 주입하므로, `.env`가 외부 시크릿 관리 도구와 Hermes를 잇는 통로 역할을 합니다. Bitwarden Secrets Manager 등 외부 시크릿 매니저에서 주입된 키는 `hermes model` 출력과 설정 마법사에서 출처가 함께 표시됩니다(예: `sk-ant-... ✓ (from Bitwarden)`). 1Password·HashiCorp Vault 등도 같은 방식으로 연동할 수 있습니다.

개별 항목에 위 4단계와 다른 우선순위가 적용되는 경우(예: `agent.max_turns`)에는 해당 항목 설명에서 명시합니다.

config.yaml에서 환경 변수 참조

config.yaml 안에서 `${ENV_VAR}` 구문으로 `.env`의 값을 참조할 수 있습니다. API 키처럼 시크릿을 `.env`에 한 곳만 두고 config.yaml에서 가져다 쓸 때 유용합니다.

```
auxiliary:
  vision:
    api_key: ${VISION_API_KEY}    # .env의 VISION_API_KEY 값으로 치환
model:
  default: ${MY_DEFAULT_MODEL}    # .env의 MY_DEFAULT_MODEL 값으로 치환
```

해당 환경 변수가 정의되어 있지 않으면 `${VAR}` 문자열이 그대로 남으므로, 반드시 `.env`에 해당 변수가 있어야 합니다.

설정 관리 명령어

```
hermes config show          # 현재 설정 보기
hermes config edit          # 에디터에서 config.yaml 편집
hermes config set KEY VAL   # 특정 값 설정
hermes config check         # 누락된 설정 확인
hermes config migrate       # 누락된 설정을 대화형으로 추가 (커스텀 엔드포인트 설정 시 서버를 자동 탐색하여 사용 가능한 모델을 감지)
hermes config path          # config.yaml 파일 경로 출력
hermes config env-path      # .env 파일 경로 출력
```

모델과 라우팅

모델 관련 설정은 한 곳에 모아 다룹니다. 이 안에서 주 모델, 라우팅, 폴백, 크리덴셜 풀, 보조 모델, 서브에이전트, MoA 순으로 다룹니다.

모델 설정

프로바이더 결정 방식은 「클라우드 API 연동」 절을 참고하세요. 모델명의 슬래시 접두사(`anthropic/`, `openai/`)는 모델 식별자이며 라우팅에 관여하지 않습니다. 어디로 요청이 가는지는 `model.provider`와 `model.base_url`이 결정합니다.

로컬 모델(Ollama)을 사용하는 경우 — `provider: "custom"`과 `base_url`을 같이 지정합니다(자세한 설정은 부록 「로컬 LLM 가이드」 참고):

```
model:
  default: "hermes3"
  provider: "custom"
  base_url: "http://<Ollama-서버-IP>:11434/v1"
```

클라우드 API를 직접 연결하는 경우 — `provider`를 명시해야 합니다.

```
model:
  default: "anthropic/claude-sonnet-4-6"       # 기본 모델
  provider: "anthropic"                         # Anthropic API 직접 연결
```

OpenRouter 경유 (기본값) — `provider` 없이 모델명만 지정하면 OpenRouter를 통해 요청됩니다.

```
model:
  default: "anthropic/claude-sonnet-4-6"       # OpenRouter 경유
```

OpenRouter 프로바이더 라우팅

OpenRouter를 프로바이더로 사용할 때, 같은 모델(예: `claude-sonnet-4`)을 호스팅하는 여러 인프라 제공자(Anthropic 직접, AWS Bedrock 등) 중 어디로 보낼지를 제어할 수 있습니다. `config.yaml`의 `provider_routing` 블록에 지정합니다.

```
provider_routing:
  sort: "throughput"          # price | throughput | latency
  order:                      # 제공자 우선순위 (선택)
    - anthropic
    - google
  data_collection: "deny"     # 데이터 보존·학습하는 제공자 제외
  require_parameters: true    # 요청의 모든 파라미터를 지원하는 제공자만 사용
```

옵션	설명
`sort`	`price`(최저가), `throughput`(처리량), `latency`(응답 속도) 중 선택. 로드밸런싱을 해제하고 기준에 따라 순서대로 시도합니다
`order`	특정 제공자를 우선 배치. 배열 순서대로 시도합니다
`data_collection`	`"deny"`로 설정하면 프롬프트·응답을 학습·보존하는 제공자를 제외합니다
`require_parameters`	`true` 이면 요청에 포함된 모든 파라미터(예: tool calling, JSON mode)를 지원하는 제공자만 사용합니다

> 💡 ***프라이버시가 중요하다면*** *`data_collection: "deny"` 를 권합니다. 업무 데이터나 개인정보가 포함된 프롬프트가 제3자 제공자에 보존되는 것을 방지합니다.*

이 설정은 **어떤 모델을 쓸지**(모델 라우팅)가 아니라, **이미 정해진 모델을 어떤 인프라에서 실행할지**(제공자 라우팅)를 결정합니다. 모델 선택과 비용 최적화 전략은 「클라우드 API 연동」 절을 참고하세요.

폴백 모델

주 모델이 실패하면 자동으로 폴백 모델로 전환됩니다. 단일 모델을 지정하는 기존 형식도 작동합니다.

```
fallback_model:
  provider: custom
  model: hermes3
  base_url: http://<Ollama-서버-IP>:11434/v1
  api_key_env: OLLAMA_API_KEY
```

여러 프로바이더를 순서대로 시도하는 체인 형식도 지원합니다. 주 모델이 실패하면 목록의 첫 번째 프로바이더를 시도하고, 그것도 실패하면 다음 프로바이더로 넘어갑니다. 체인이 모두 소진될 때까지 순서대로 시도합니다.

```
fallback_providers:
  - provider: openrouter
    model: anthropic/claude-sonnet-4
  - provider: openai
    model: gpt-4o
```

`fallback_providers`가 설정되어 있으면 `fallback_model`보다 우선합니다.

크리덴셜 풀

같은 프로바이더에 여러 API 키를 등록해 두면, 요청이 429(속도 제한)·401·402 오류로 실패할 때 자동으로 다음 키로 전환합니다. 여러 계정의 할당량을 합산하여 사용할 수 있어 속도 제한에 걸리는 빈도를 줄여 줍니다.

크리덴셜 풀은 CLI 명령어로 관리합니다.

```
hermes auth add       # 프로바이더에 API 키 추가 등록
hermes auth list      # 등록된 크리덴셜 목록 조회
hermes auth remove    # 크리덴셜 삭제
hermes auth reset     # 풀 상태 초기화
```

보조 모델

이미지 분석, 웹 페이지 요약, 컨텍스트 압축 등 부수적인 작업에는 별도의 경량 모델을 지정할 수 있습니다. 보조 모델 슬롯은 9개입니다.

```
auxiliary:
  vision:           # 이미지 분석
    provider: "auto"
    model: ""       # 비어 있으면 주 모델 사용
  web_extract:      # 웹 페이지 요약
    provider: "auto"
    model: ""
  # 그 외: compression(컨텍스트 압축), session_search(세션 검색),
  #        skills_hub(스킬 허브), approval(위험 명령 판단),
  #        mcp(MCP 도구), title_generation(세션 제목 생성),
  #        curator(스킬 라이브러리 큐레이터)
```

각 슬롯에 provider, model, base_url, api_key를 개별 지정할 수 있습니다. 비워두면 주 모델을 사용합니다. `curator` 슬롯은 백그라운드로 스킬을 정리하는 자동 큐레이터에 쓰이며, 라이브러리가 클수록 토큰 소비가 커지므로 보통 저렴한 보조 모델을 지정합니다(자세한 내용은 7장 참고).

OpenRouter를 보조 모델 프로바이더로 사용할 때는 `extra_body`로 제공자 라우팅이나 Pareto Code 라우터를 슬롯별로 개별 지정할 수 있습니다. 주 모델의 `provider_routing`과 `openrouter.min_coding_score`는 보조 작업에 전파되지 않으므로, 필요하면 슬롯마다 따로 설정해야 합니다.

```
auxiliary:
  compression:
    provider: "openrouter"
    model: "openrouter/pareto-code"
    extra_body:
      plugins:
        - id: pareto-router
          min_coding_score: 0.5
```

서브에이전트 위임 모델

`delegate_task` 도구로 하위 작업을 위임할 때, 주 모델과 다른 모델을 사용할 수 있습니다.

```
delegation:
  model: "llama3.1:8b"
  provider: "custom"
```

이렇게 하면 주 모델(14B)이 복잡한 판단을 하고, 서브에이전트(8B)가 단순 작업을 처리하는 **비용 최적화 구성**이 가능합니다.

Mixture of Agents (MoA)

여러 프론티어 모델을 동시에 호출하고 결과를 종합하는 고급 기능도 있습니다. OpenRouter 계정이 필요하며, 로컬 LLM만 사용하는 환경에서는 해당하지 않습니다.

> 멀티 모델 전략
>
> 한국어 중심으로 사용한다면, 주 모델을 클라우드 API(Claude Sonnet 등)로 설정하고 보조 모델 슬롯에 Gemini Flash 같은 저렴한 클라우드 모델을 배정하는 구성이 현실적입니다. 서브에이전트(`delegation.model`)에 경량 모델을 지정하면 단순 작업의 비용을 줄일 수 있습니다. 세션 도중 `/model`로 모델을 전환하는 방법과 비용 최적화 전략은 「클라우드 API 연동」 절에서 다룹니다.

그 밖의 주요 설정 항목

터미널 백엔드

```
terminal:
  backend: local          # local, docker, ssh, singularity, modal, daytona, vercel_sandbox
  cwd: "."                # 작업 디렉터리
  timeout: 180            # 명령 타임아웃 (초)
  persistent_shell: true  # 셸 세션 유지
```

메모리 설정

```
memory:
  memory_enabled: true
  user_profile_enabled: true
  memory_char_limit: 2200      # 약 800 토큰
  user_char_limit: 1375        # 약 500 토큰
```

컨텍스트 압축

```
compression:
  enabled: true
  threshold: 0.50              # 컨텍스트의 50%에서 압축 시작
  summary_model: "llama3.1:8b" # 요약에 사용할 모델
```

프롬프트 캐싱 (Anthropic)

Anthropic 계열 모델로 같은 시스템 프롬프트·툴 정의·대화 앞부분을 반복해서 보낼 때, 캐시된 토큰으로 비용을 줄이는 기능입니다. 기본 TTL은 5분이며, 같은 세션에서 캐시가 자주 재사용되는 패턴(긴 대화·반복 호출)이면 1시간으로 늘려 캐시 적중률을 높일 수 있습니다.

```
prompt_caching:
  cache_ttl: "5m"   # "5m"(기본) 또는 "1h"
```

`5m`과 `1h` 외 값은 무시되고 5분이 유지됩니다. CLI 시작 시 `💾 Prompt caching: ENABLED (..., 5m TTL)` 형태로 현재 TTL이 표시됩니다.

v0.14.0부터 1시간 TTL이 **세션 경계를 넘어** 유지됩니다. `/new`로 새 세션을 시작해도 직전 세션의 시스템 프롬프트·스킬·메모리 프리픽스 캐시가 살아 있어, 첫 응답을 더 빠르고 저렴하게 받습니다. 백그라운드 메모리 리뷰도 같은 캐시를 사용해 매 턴 풀 가격을 치르지 않습니다. Anthropic·OpenRouter·Nous Portal 경유 호출에 모두 적용됩니다.

보안 — 시크릿 마스킹

도구 출력·로그에 노출되는 API 키·OAuth 토큰 같은 시크릿 패턴을 자동으로 `[REDACTED]`로 가리는 기능입니다. **기본은 OFF**이며, 패치/도구 출력에 시크릿 모양의 가짜 일치가 끼어들어 결과를 망가뜨리는 사고를 막기 위해 명시적 opt-in입니다.

```
security:
  redact_secrets: false   # 로그·도구 출력의 시크릿 자동 마스킹
```

`true`로 켜면 환경 변수 `HERMES_REDACT_SECRETS=true`로 브리지되어 redaction이 활성화됩니다. 화면 출력에서 항상 가려야 하는 자리(상태 배너 등)는 이 설정과 무관하게 강제 마스킹됩니다.

보안 — lazy-install 및 네트워크

```
security:
  allow_lazy_installs: true    # 선택적 의존성의 런타임 자동 설치 허용 (c1eb2dcda)

network:
  force_ipv4: false            # IPv6가 고장난 서버에서 IPv4 강제 사용
```

표시 및 입력

```
display:
  busy_input_mode: interrupt   # interrupt, queue 또는 steer
  busy_ack_enabled: true       # 작업 중에 받은 입력을 메시징 채널에 ⚡/⏳/⏩로 알릴지
  streaming: true              # 스트리밍 응답 활성화
  show_reasoning: false        # 모델의 추론 과정(thinking) 표시
  bell_on_complete: false      # 작업 완료 시 터미널 벨
```

`busy_input_mode`는 에이전트가 작업 중일 때 Enter 키의 작동을 결정합니다.

값	작동
`interrupt`	현재 작업을 중단하고 새 입력을 처리 (기본값)
`queue`	입력을 대기열에 넣고 현재 작업이 끝난 후 처리
`steer`	에이전트가 다음 도구 호출을 마친 뒤 메시지를 주입. 인터럽트 없이 프롬프트 캐시를 유지하면서 방향을 전환할 수 있습니다(`635253b91`)

Ctrl+C는 설정과 무관하게 항상 중단됩니다.

`busy_ack_enabled`는 게이트웨이(텔레그램·디스코드 등)에서 작업 중일 때 사용자가 추가 메시지를 보내면 ⚡(인터럽트), ⏳(큐), ⏩(스티어) 같은 ack 메시지를 채널에 회신할지 여부입니다. 음성 입력이나 빠른 연속 메시지에서 ack가 시끄럽게 느껴지면 `false`로 끕니다. `false`로 둬도 입력 자체는 정상적으로 큐잉/스티어/인터럽트되며, 채널의 ack 회신만 억제됩니다. 환경 변수 `HERMES_GATEWAY_BUSY_ACK_ENABLED`로도 제어할 수 있습니다.

에이전트 작동

```
agent:
  reasoning_effort: "medium"   # xhigh/high/medium/low/minimal/none
  max_turns: 60                # 대화당 최대 반복 횟수
```

`reasoning_effort`의 설정 파일 기본값은 `"medium"`입니다. `max_turns`는 설정 파일(`cli-config.yaml.example`)에서 60으로 제공되며, 코드 레벨 폴백은 90입니다. CLI에서 `--max-turns` 플래그로 일시적으로 덮어쓸 수 있습니다. 적용 우선순위는 CLI 플래그 > `agent.max_turns` > 환경 변수 `HERMES_MAX_ITERATIONS` > 코드 기본값 90입니다.

마이그레이션 노트

버전이 올라가면서 기존 설정 방식이 바뀐 항목을 한 곳에 모았습니다. 이전 버전에서 올라오는 독자는 이 절을 먼저 훑어 보면 좋습니다.

v0.7.0 — `OPENAI_BASE_URL` 환경 변수 무효화

`.env`의 `OPENAI_BASE_URL` 환경 변수는 더 이상 라우팅에 참여하지 않습니다. 엔드포인트는 `config.yaml`의 `model.base_url`이 단일 출처입니다. 프로바이더를 바꿀 때는 `.env`에 키만 추가해도 라우팅이 바뀌지 않으므로, `hermes model`로 새 프로바이더를 선택하거나 `model.{provider,base_url,default}`를 같이 갱신해야 합니다(자세한 함정은 「클라우드 API 연동」 절 참고).

v0.13.0 — `smart_model_routing` 제거

메시지 복잡도에 따라 cheap_model과 기본 모델을 자동 분배하던 기능이 v0.13.0에서 제거됐습니다(`424e9f36b`). 대체 경로로 `openrouter/pareto-code` 라우터가 도입됐으며, 비용 최적화 전략은 「클라우드 API 연동」 절을 참고합니다.

설정 진단

문제가 의심될 때는 `hermes doctor`를 실행합니다. Python 버전, 의존성 상태, 설정 파일 유효성, API 연결 상태 등을 종합적으로 점검해 줍니다.

```
hermes doctor
```

02-6. 업데이트

- 자주 쓰는 플래그
- 업데이트 과정
  - 설정 마이그레이션 단계
- 복원 여부 선택
- 충돌이 발생하면
- HERMES_HOME 사전 백업
- 번들 스킬과 업데이트

Hermes Agent는 활발히 개발되고 있어 업데이트가 잦습니다. CLI를 실행하면 배너에 새 커밋 수가 표시되고, 다음 명령어로 업데이트합니다.

```
hermes update
```

대화 중이라면 슬래시 명령어 `/update`로도 실행할 수 있습니다.

자주 쓰는 플래그

플래그	작동
`--check`	새 업데이트가 있는지만 확인하고 실제 설치는 하지 않습니다. CI나 배너에서 미리 확인할 때 유용합니다.
`--yes` / `-y`	대화형 프롬프트(설정 마이그레이션, stash 복원)에 자동으로 Yes로 답합니다. API 키 입력은 건너뛰므로, 필요하면 `hermes config migrate`를 따로 실행하세요.
`--backup`	이번 한 번만 업데이트 직전에 HERMES_HOME 백업을 강제합니다(아래 백업 절 참고).
`--no-backup`	설정으로 켜져 있어도 이번 한 번은 백업을 건너뜁니다. `--no-backup`이 `--backup`을 이깁니다.

업데이트 과정

내부적으로 `hermes update`는 다음 순서로 작동합니다.

1. `git fetch`로 원격 저장소의 새 커밋을 확인합니다.

2. 로컬에 수정된 파일이 있으면 자동으로 `git stash`에 임시 저장합니다.
3. `git pull`로 새 커밋을 가져옵니다.
4. **구문 검증 + 자동 롤백** — `hermes` 실행 시 항상 임포트되는 핵심 파일들(`hermes_cli/main.py`, `config.py`, `cli.py`, `run_agent.py` 등 8개)을 `py_compile`로 컴파일해 봅니다. 한 곳이라도 구문 오류가 있으면(예: 머지 충돌 마커가 남은 잘못된 커밋이 main에 올라온 경우) pull 직전 SHA로 `git reset --hard`해서 자동 롤백합니다. CLI가 부팅 불가 상태로 남는 것을 방지하는 안전장치입니다. 사용자는 잠시 후 수정 커밋이 올라오면 다시 `hermes update`를 시도하면 됩니다.
5. **설정 마이그레이션** — 새 버전에서 추가된 설정 항목이 있는지 점검하고, 필요하면 대화형으로 채워 넣습니다.
6. stash한 변경사항을 복원할지 물어봅니다.

설정 마이그레이션 단계

새 버전에서 추가된 설정 키가 있고 사용자의 `~/.hermes/config.yaml`에 그 키가 없으면, 업데이트 중에 대화형으로 추가할지 물어봅니다. 이는 별도 명령인 `hermes config migrate`와 같은 작업이며, 업데이트 중에는 그 단계를 인라인으로 묶어 한 번에 진행하는 것입니다.

- **추가**: 새 항목을 채워 넣고 싶으면 `Y`로 답합니다. 키마다 안내가 뜨고, 비워 두면 내장 기본값이 그대로 적용됩니다.
- **건너뛰기**: 나중에 처리하고 싶으면 `n`으로 답합니다. 새 항목은 코드 레벨 기본값으로 작동하며, 나중에 `hermes config migrate`를 따로 실행해 채울 수 있습니다.
- **자동 yes(`--yes`)**: 모든 대화형 항목에 자동으로 `Y`를 보냅니다. 단, API 키처럼 사용자 입력이 필요한 자리는 비워 두므로, 그 부분은 이후 `hermes config migrate`로 따로 채워야 합니다.

기존 설정 키는 그대로 보존됩니다. 폐기된 키(예: v0.13에서 제거된 `smart_model_routing`)는 설정에 남아 있어도 무시되며, 마이그레이션은 새 키 추가만 다룹니다.

```
→ Local changes detected — stashing before update...
→ Found 71 new commit(s)
→ Pulling updates...

⚠ Local changes were stashed before updating.
  Restoring them may reapply local customizations onto the updated codebase.
  Review the result afterward if Hermes behaves unexpectedly.
Restore local changes now? [Y/n]
```

복원 여부 선택

선택	언제
Y (기본값)	스킬이나 설정 파일을 직접 수정한 적이 있을 때. 대부분의 경우 여기에 해당합니다.
n	로컬 변경이 더 이상 필요 없거나, 충돌 가능성이 높아 나중에 직접 확인하고 싶을 때.

`n`을 선택해도 변경사항은 사라지지 않습니다. git stash에 보관되어 있으므로 나중에 복원할 수 있습니다.

```
git stash list                # 저장된 stash 목록 확인
git stash apply stash@{0}     # 가장 최근 stash 복원
git stash drop stash@{0}      # 복원 후 stash 삭제
```

충돌이 발생하면

`Y`를 선택했는데 업데이트된 코드와 로컬 변경이 같은 파일을 수정한 경우, 충돌이 발생할 수 있습니다. 이때 Hermes는 충돌 파일 목록을 보여주고 작업 디렉터리를 초기화할지 물어봅니다. 로컬 변경이 stash에 남아 있으므로 초기화해도 작업이 유실되지 않습니다.

충돌을 해결한 뒤에는 stash 항목을 정리합니다.

```
git stash drop stash@{0}      # 사용한 stash 삭제
```

> 로컬 변경을 줄이는 방법
>
> 설정은 `~/.hermes/config.yaml`과 `~/.hermes/.env`에 분리되어 있습니다. 소스 코드를 직접 수정하는 대신 이런 공식 확장 포인트를 활용하면 업데이트 시 충돌을 피할 수 있습니다.

HERMES_HOME 사전 백업

업데이트 직전에 `~/.hermes/`(설정·메모리·스킬 등)를 통째로 압축 백업해 두는 옵션입니다. 기본은 OFF이며, 매 업데이트마다 백업을 만들어 두려면 `config.yaml`에서 켭니다.

```
updates:
  pre_update_backup: true    # 업데이트 직전 자동 백업 생성
  backup_keep: 5             # 보관할 백업 개수 (오래된 것부터 삭제)
```

체크포인트 데이터(`checkpoints/`)와 SQLite의 WAL·SHM·저널 사이드카는 백업에서 제외됩니다 — 빠르게 비대해지고 복원 가치가 낮기 때문입니다. 한 번만 강제하거나 끄려면 위의 `--backup` / `--no-backup` 플래그를 사용합니다.

번들 스킬과 업데이트

`~/.hermes/skills/`에 설치되는 번들 스킬은 `hermes update` 시 자동으로 동기화됩니다. 이때 사용자가 수정한 스킬은 보호됩니다.

상황	작동
새 버전에 추가된 스킬	자동으로 복사
사용자가 수정하지 않은 스킬	번들 최신 버전으로 업데이트
사용자가 수정한 스킬	**건너뜀** (수정 사항 유지)
사용자가 삭제한 스킬	다시 추가하지 않음

동기화 결과는 업데이트 시 표시됩니다.

```
→ Syncing bundled skills ...
  + 3 new: new-skill-a, new-skill-b, new-skill-c
  ↑ 2 updated: arxiv, github-pr-workflow
  ~ 1 user-modified (kept): hermes-agent
```

내부적으로 `~/.hermes/skills/.bundled_manifest` 파일에 각 스킬의 원본 해시가 기록되어 있어, 사용자 수정 여부를 자동으로 판별합니다. 에이전트가 스킬을 자동 개선(`skill_manage edit`)한 경우도 사용자 수정으로 취급되어 보호됩니다.